GUIDE · BUILT FROM THE PROJECT DOCUMENTS

The wsjtx-mcp guide

wsjtx-mcp handles the weak-signal modes. It is one of three connectors, with fldigi-mcp and n3fjp-mcp, that operate and then log. This guide covers what the WSJT-X UDP protocol can and cannot do, installing, a first session, and reaching WSJT-X on another computer. The message-by-message reference is WSJTX-API.md.

VERSION	MODES	PROTOCOL
v0.1.4	FT8 FT4 JT65 MSK144 Q65 WSPR	UDP schema 3 · port 2237

VERIFIED AGAINST
WSJT-X 3.0.2

At a glance

WSJT-X has no API where you send a request and get a response. It broadcasts its state over UDP in messages such as Status, Decode, QSOLogged and Heartbeat. It also accepts a small set of control messages. wsjtx-mcp runs a UDP listener in the background that reads each datagram as it arrives. The listener keeps the latest status, a buffer of decodes and the completed QSOs. You read those, and you steer WSJT-X with control messages.

This has four consequences for how you operate.

- **wsjtx-mcp cannot change the dial frequency.** You can read the dial frequency from Status. With configure you can set the mode, sub-mode, Rx DF and T/R period. To QSY, use rig control: Hamlib, CAT or the WSJT-X window. This server does not do it.
- **You can start and halt a transmission, but you cannot switch Enable Tx on or off.** A transmission starts when you answer a CQ with reply, or when you send free text with

send. halt stops it. The Enable Tx, Auto Seq, Call 1st and Hold Tx Freq checkboxes can only be set in the WSJT-X window.

- **reply gives hands-free QSOs only when Auto Seq is on.** A reply does the same as double-clicking a CQ. Auto Seq is usually on by default for FT8 and FT4. With it on, WSJT-X runs the whole exchange through to QSOLogged with no further calls. With Auto Seq off, reply starts only the first transmission.
- **It suits search and pounce better than running.** The API is built to answer CQ and QRZ decodes, so search and pounce can be fully automated. There is no clean way to call CQ over and over. For that you need WSJT-X's own Enable Tx, or a free-text CQ sent again each period.

The protocol is schema 3 (Qt_5_4). It has not changed from WSJT-X 2.1 through 3.x. wsjtx-mcp has been checked against a running WSJT-X 3.0.2.

Installing

1 · Enable WSJT-X's UDP Server

In WSJT-X, open **Settings → Reporting → UDP Server**. Set UDP Server to the host that runs wsjtx-mcp. If both run on the same computer, leave it at `127.0.0.1`. Leave the port at 2237, the default. **Tick Accept UDP requests** if you want wsjtx-mcp to control WSJT-X, for example to answer CQs, set free text or change the configuration. It is off by default. Without it, wsjtx-mcp can still read decodes and status, but it cannot send commands.

2a · As a desktop connector in Claude

01 Download `wsjtx-mcp.mcpb` from Downloads.

02 If you installed an older build before, remove it first. Go to **Settings → Desktop app → Extensions** (click **Manage**), uninstall it, and wait for its tile to disappear. Then the new build can take its place.

03 In Claude, open **Settings → Desktop app**, click **Manage** next to **Extensions**, and drag `wsjtx-mcp.mcpb` onto that page. If dragging does not work, click **Advanced settings**, then **Install extension**, and choose the file. The new connector then shows under **Settings → Customize → Connectors**.

04 Fill in the settings form. Enter your operator callsign to allow transmitting, or leave it blank to receive only. The listen host and port are usually `127.0.0.1` and 2237.

05 Quit and reopen Claude Desktop (⌘Q) so the new tools load.

> **ENABLES TRANSMIT**
> Entering your callsign, together with Accept UDP requests in WSJT-X, lets wsjtx-mcp key the transmitter through `reply` and `free_text` with `send=true`. Leave the callsign blank until you have read the transmit gate chapter.

2b · From PyPI, any MCP client (advanced)

This is for advanced users. If you use Claude Desktop, the connector in 2a is all you need.

> **UNTESTED OUTSIDE CLAUDE**
> MCP is an open standard, so other assistants and agentic harnesses such as OpenClaw should work too. They have not been tested, and there are no ready-made configurations for them yet.

```
uvx wsjtx-mcp        # or: pipx run wsjtx-mcp
```

Add it to your client's MCP configuration with the environment variables from the table under How it works. For example:

```
{
  "mcpServers": {
    "wsjtx-mcp": {
      "command": "uvx",
      "args": ["wsjtx-mcp"],
      "env": { "WSJTX_CALLSIGN": "", "WSJTX_HOST": "127.0.0.1", "WSJTX_PORT": "2237
    }
```

```
  }
}
```

3 · Verify

Ask your client to run the status tool. Within about 15 seconds you should see the dial frequency, the mode and the WSJT-X instance it found. WSJT-X sends a Heartbeat every 15 seconds and a Status each time its state changes. If nothing appears, run diagnostics. It reports whether the UDP listener could bind its port and how many datagrams have arrived.

Your first session

```
What is WSJT-X doing right now?
    status: dial frequency, mode, DX call, transmit state, instance health
Show me the last few decodes on this band.
    decodes read: the buffered Band Activity
Answer the strongest CQ on this band.
    reply; with Auto Seq on, WSJT-X runs the whole exchange to QSOLogged
Log that contact to N3FJP.
    the log tool returns the buffered QSO; the assistant passes it to n3fjp-mcp
stop
    transmit halt; always allowed
```

N3FJP runs only on Windows. On a Mac it runs in a Windows virtual machine or on a Windows PC, as Where everything runs shows.

KEYS THE TRANSMITTER

Answering a CQ puts the station on the air. If `WSJTX_CALLSIGN` is blank, wsjtx-mcp refuses the request. Under Part 97 you are still the control operator. Stay where you can step in, and say "stop" to halt.

How it works

For the most part, WSJT-X broadcasts. It sends Status, Decode, QSOLogged, LoggedADIF, WSPRDecode, Clear, Close and a Heartbeat every 15 seconds. There is no "get frequency" or "get mode" query. You learn its state from Status.

The listener keeps three things current. The first is the latest Status snapshot. The second is a decode buffer you can read, drain, clear or replay. The third is the completed QSOs from QSOLogged and LoggedADIF messages, ready to feed the logger.

Control works only when Accept UDP requests is on. Every control message must carry the Id of the WSJT-X instance it is meant for. The server learns that Id from the first datagram the instance sends. Control replies go back to the address each datagram came from. The host and port in the settings are where wsjtx-mcp listens.

Three points from the message reference affect how you use it.

- `reply` must match an earlier CQ or QRZ decode in full. It does the same as double-clicking that line.
- `configure` sets mode, sub-mode, Rx DF, T/R period, frequency tolerance, DX call and grid, but not the dial frequency. On WSJT-X 3.0.x a mode change may move the dial to that band and mode's default frequency.
- Only one process can bind a UDP port. That rule comes from UDP and the operating system, not from WSJT-X.

The codec uses only the Python standard library and handles all 17 message types.
The details of each message are in WSJTX-API (PDF).

VARIABLE	DEFAULT	PURPOSE
`WSJTX_HOST`	`127.0.0.1`	UDP address to bind and listen on.
`WSJTX_PORT`	`2237`	WSJT-X UDP Server port.
`WSJTX_CALLSIGN`	empty	Operator callsign. It is the only setting that allows transmitting. Blank means receive only.

VARIABLE	DEFAULT	PURPOSE
`WSJTX_MULTICAST`	off	Optional multicast group to join, so wsjtx-mcp can share the traffic with other UDP programs.
`WSJTX_INSTANCE`	auto	Target a specific WSJT-X Id when several instances broadcast.

The transmit gate

> **CAN KEY A TRANSMITTER**
>
> The callsign is the only setting that allows wsjtx-mcp to transmit. If `WSJTX_CALLSIGN` is blank, the server only receives. It refuses every message that would start a transmission. Those are `reply`, `free_text` with `send=true`, and any keying message sent through `wsjtx_call`. Halt, clear, configure, highlight, location, replay and all reads always work. None of them puts you on the air, and halt takes you off. Your client's tool-permission prompt also asks you to approve each transmission. WSJT-X's own Tx Watchdog, and the Tx Enabled and Transmitting flags in status, give you more safety signals. As the licensed control operator, you are responsible for lawful operation. That means station identification, being able to step in, and following the Part 97 rules for automatic and remote control.

The tools

TOOL	KIND	WHAT IT DOES
`status`	observe	Latest `Status` snapshot, plus the health of the listener and the WSJT-X instance.
`diagnostics`	observe	Host and network, whether the port is bound, datagram counts and the transmit gate state.
`decodes`	observe/nudge	`read`, `drain` (new decodes only), `clear_local` or `replay`. This is the receive side.

TOOL	KIND	WHAT IT DOES
`log`	observe	Completed QSOs from the buffer (`QSOLogged` and `LoggedADIF`), ready to pass to N3FJP.
`reply`	**transmit**	Answer a buffered CQ or QRZ decode. When WSJT-X "Auto Seq" is on, WSJT-X sequences the rest of the QSO.
`free_text`	**transmit** if `send`	Set the Tx5 free-text message. `send=true` keys the radio.
`transmit`	control	`halt` or `halt_auto` stops transmitting. UDP cannot *enable* Tx.
`configure`	control	Set mode, sub-mode, Rx DF, T/R period, frequency tolerance, DX call and grid. **Not the dial frequency.**
`clear`	control	Clear the Band Activity / Rx Frequency windows.
`highlight`	control	Color or clear a callsign in Band Activity.
`location`	control	Override the session Maidenhead grid.
`switch_config`	control	Switch to a named WSJT-X configuration.
`annotate`	control	Set a Fox/Hound sort-order annotation for a DX call. Rarely needed outside DXpeditions.
`wsjtx_call`	escape hatch	Build and send any message type by name. The transmit gate still applies.

Running alongside other UDP tools

Only one process on a computer can own UDP port 2237. If JTAlert, GridTracker or N1MM already uses it, you have three options.

01 Point WSJT-X's secondary UDP server at wsjtx-mcp's host and port.

02 Use a multicast group. Set the same group in WSJT-X's UDP Server and in `WSJTX_MULTICAST` , and several listeners can share it.

03 Run wsjtx-mcp on a different computer from WSJT-X and connect the two with mcp-host-bridge. The next section explains how.

The remote host

Some MCP clients run their connectors in a sandbox, and Claude Desktop is one of them. Its connectors can reach only `127.0.0.1`. They cannot reach LAN addresses, even with the macOS Local Network permission turned on. To get around this, run mcp-host-bridge, a small UDP relay, outside the sandbox on the wsjtx-mcp computer. It passes WSJT-X's broadcasts in and your control replies back out. UDP has no connections and traffic flows both ways, so the setup is the reverse of the TCP case. Here the bridge listens on the LAN and delivers to loopback.

You can get the bridge with Python. You can also download the single binary for your operating system from Downloads. The binary needs no Python.

```
uvx mcp-host-bridge --help          # or: pipx install mcp-host-bridge
```

Then install the `wsjtx` bridge as a persistent service on the wsjtx-mcp computer. The same commands work on every operating system. They use launchd on macOS, systemd on Linux and a Scheduled Task on Windows. `netsh portproxy` only handles TCP, so the bridge skips it for UDP.

```
mcp-host-bridge install wsjtx --to <remote-wsjtx-host>
# listens 0.0.0.0:2237 (LAN) and delivers 127.0.0.1:2238 (loopback);
# --to is an optional hint; the remote peer is auto-learned from the first datagram
mcp-host-bridge status wsjtx       # check it
mcp-host-bridge uninstall wsjtx    # remove it
```

With the bridge in place, wsjtx-mcp must use `WSJTX_HOST=127.0.0.1` and `WSJTX_PORT=2238`. Port 2238 is where the bridge delivers. Do not use the default 2237, because the bridge's own LAN socket already holds 2237 on that computer. In the remote WSJT-X, set Settings → Reporting → UDP Server to the LAN IP address of the bridge computer, port 2237. Tick Accept UDP requests if you want control.

If WSJT-X runs on the same computer as wsjtx-mcp, you do not need the bridge. Point wsjtx-mcp at `127.0.0.1:2237`. If every program that listens is on the same LAN segment, multicast can replace the bridge. Set `WSJTX_MULTICAST` and WSJT-X's UDP Server to the same group. The full details are in REMOTE-HOST.md.

AE5VG · AUSTIN, TX

Site text CC BY 4.0; software under the license named in each repository.

GL es 73 de AE5VG sk

CONTACT

Software: open an issue on the relevant repository at github.com/sbrunner-atx.

LinkedIn: Stefan Brunner.

Email: stefan@ae5vg.com